Labo 4 - Memory management

Robin De Vos (MEIECFS)
Anthony Tacquet (MEISWE)

March 2026



1 Context

For simplicity in both part 1 and 2 of this lab exercise we decided to use only square matrices.

1.1 Part 1 - Coalesced memory access

The goal of part 1 of this lab is to show the idea of coalesced memory access, we can show this by comparing the copy and transpose array-operation execution times.

Since the copy operation makes threads within warps access more or less the same memory group, less delay will occur in comparison with the transpose operation, which will make threads within a warp access different memory groups (more noticeable for larger arrays).

A comparison between the operation times on the GPU and CPU is also required for this part of the lab exercise, this for both the copy and the transpose operation.

1.2 Part 2 - Global, shared and constant memory

For part 2 of this lab assignment we had to compare the execution times for the multiply operation on two matrices, the comparison had to be made between using global, shared and constant memory.

1.3 Question 1: Threads migrate data from global to shared memory, where to put ___syncthreads()

You have to add ___syncthreads() at two places in code:

1. after migrating items to shared memory, to ensure shared memory is fully populated;
2. after processing the data in shared memory, to ensure all processes are done before moving the results.

1.4 Question 2: How much time does memory migration take (CPU to GPU global; GPU global to GPU shared)?

Memory migration times operate on vastly different scales depending on the hardware path:

- CPU to GPU global: takes on the order of milliseconds because data must travel off-chip across the slower PCIe bus.
- GPU global to GPU shared: takes on the order of nanoseconds to microseconds since it occurs on-chip with much higher bandwidth and lower latency.

2 Methodology

2.1 Part 1 - Coalesced memory access

For our measurements we decided to take average measurements of 20 samples and for our GPU function we used the arbitrary values of 10 Blocks and 1024 Threads. Each measurement we took was with an incremented exponent for the dimension of the array. The matrix $N * N$ with $N = 2^p$ and p can be calulated as $p_n = p_0 + \sum_{i=1}^{n} 1 = p_0 + n$ with n the max amount of iterations $n = 9$ and p_0 the initial exponent $p_0 = 1$. The code snippets can be seen in listing 1 and 2.

The CUDA code for the matrix copy is quite easy to understand, each thread has its own matrix cell that it has to copy to the other matrix.

Listing 1: GPU Copy Code

```
__global__ void copy(float *a, float *r, int n) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    int strides = n / (blockDim.x * gridDim.x);

    for (int i = 0; i < strides; i++) {
        int array_index = idx + (i * blockDim.x);

        if (array_index < n) {
            r[array_index] = a[array_index];
        }
    }
}
```

The transpose code is slightly more challenging, each thread calculates its row and column position and calculates what cell has to be copied into the resulting matrix according to the formula $index = col * N + row$ with N the dimension of the square matrix.

Listing 2: GPU Transpose Code

```
__global__ void multiply(int *a, int *b, int *r, int n, int N) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    int row_r = idx / N;
    int col_r = idx % N;
    if (idx < n) {
        for (int i = 0; i < N; i++) {
            r[idx] += a[row_r * N + i] * b[i * N + col_r];
        }
    }
}
```

2.2 Part 2 - Global, shared and constant memory

Listing 3: CUDA code for global memory mutliplication

```
__global__ void multiply(int *a, int *b, int *r, int n, int N) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    int row_r = idx / N;
    int col_r = idx % N;
    if (idx < n) {
        for (int i = 0; i < N; i++) {
            r[idx] += a[row_r * N + i] * b[i * N + col_r];
        }
    }
}
```

Listing 4 uses a synchronisation step after copying the data from global to shared memory before any operations can be performed on the shared memory data, otherwise we would have data races. Synchronisation also happens after the operation itself as mentioned in section 1.3.

Listing 4: CUDA code for shared memory multiplication

```
__global__ void multiply(int *r, int *a, int *b, int n, int N) {
    extern __shared__ int shared[];
    int idx = threadIdx.x;

    if (idx < n) {
        shared[idx]         = a[idx];        // load a
        shared[n + idx]     = b[idx];        // load b
        shared[2 * n + idx] = 0;             // zero result
           partition
    }
    __syncthreads();

    if (idx < n) {
        int row_r = idx / N;
        int col_r = idx % N;
        for (int i = 0; i < N; i++) {
            shared[2 * n + idx] += shared[row_r * N + i] * shared
               [n + i * N + col_r];
        }
    }
    __syncthreads();

    if (idx < n) {
```

```
22          r[idx] = shared[2 * n + idx];
23      }
24  }
```

Listing 5: CUDA code for constant memory multiplication

```
1  #define MAX_N 64
2  __constant__ int const_a[MAX_N * MAX_N];
3  __constant__ int const_b[MAX_N * MAX_N];
4
5  __global__ void multiply(int *r, int n, int N) {
6      int idx   = threadIdx.x + blockIdx.x * blockDim.x;
7      int row_r = idx / N;
8      int col_r = idx % N;
9      if (idx < n) {
10         for (int i = 0; i < N; i++) {
11             r[idx] += const_a[row_r * N + i] * const_b[i * N +
                   col_r];
12         }
13     }
14 }
```

3 Observations/results

For all our figures we used a log scale with base 2 on both the X and Y-axis in order to better visualise the timing data, we used base 2 since our measurements were taken with increasing powers of two.

3.1 Part 1 - Coalesced memory access

Both figure 1 and 2 show a similar trend, where initially the CPU is faster, but this changes quickly for increasing matrix sizes, while the GPU operation duration remains rather constant.

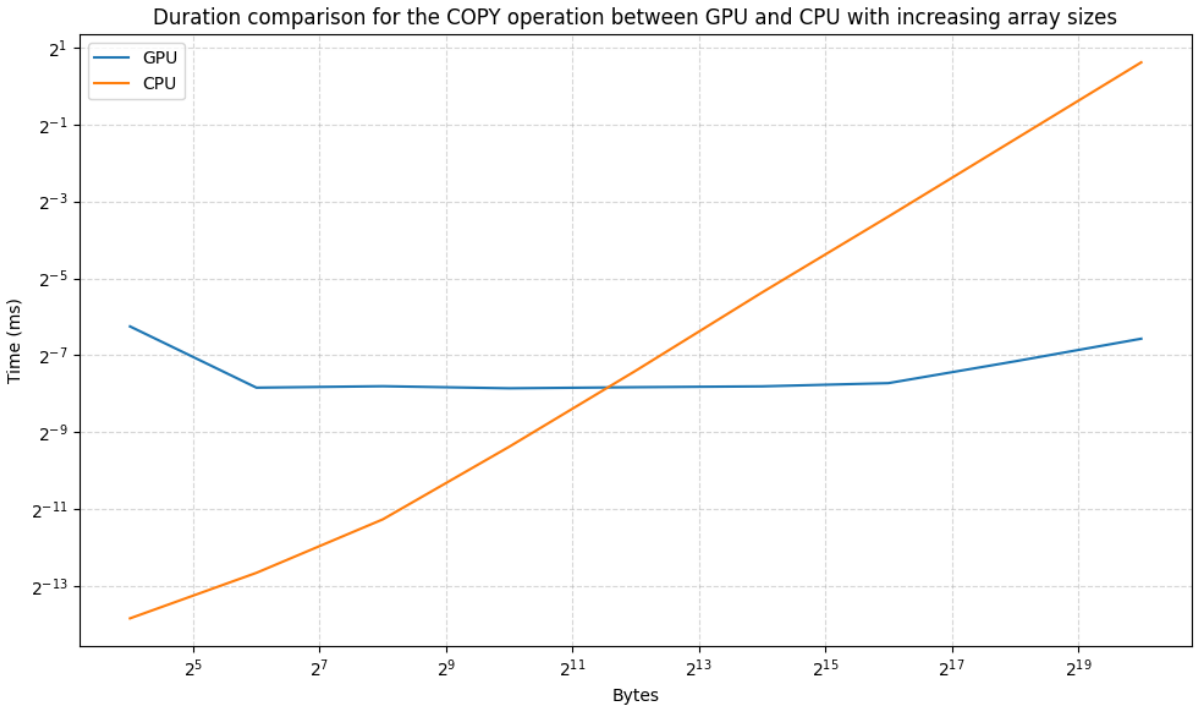


Figure 1: Duration comparison for the copy operation between GPU and CPU with increasing array sizes

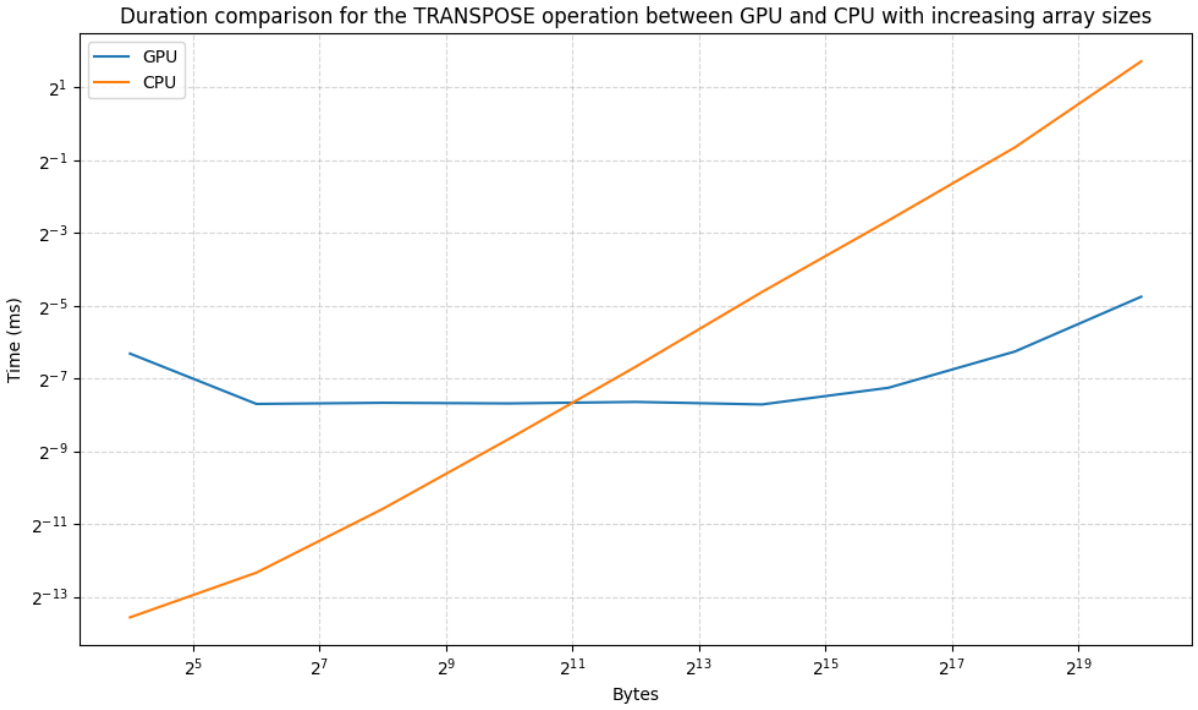


Figure 2: Duration comparison for the transpose operation between GPU and CPU with increasing array sizes

As seen in figure 3 the copy operation is at larger matrix sizes faster mostly due to

coalesced memory but also because the copy operation is an easier operation than the transpose operation even if this difference is rather small.

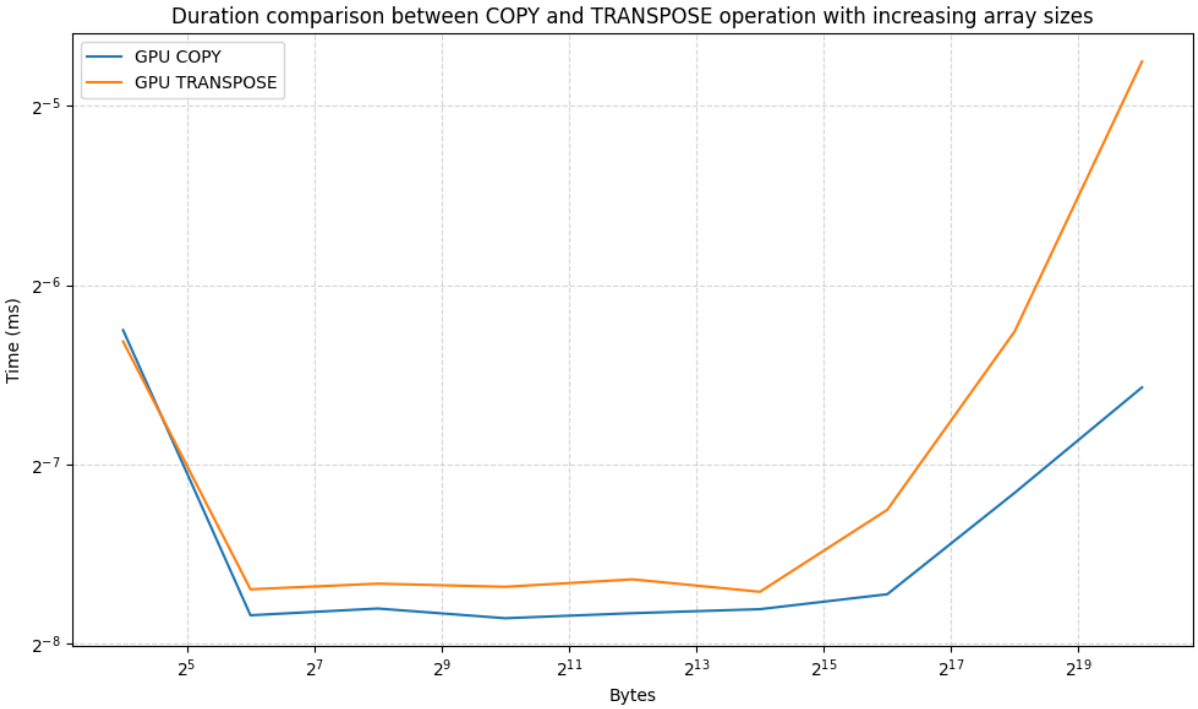


Figure 3: Operation comparison between copy and transpose operation with increasing array sizes

3.2 Part 2 - Global, shared and constant memory

In figure 4 we can clearly see that both our constant and shared memory measurement stop early in comparison with the global memory measurements. This is due to the sizes of these memories. Our constant memory has a maximum size of 64 KB shared across all threads on our CUDA device. Shared memory has a default size of 48 kB per thread block, shared across all threads within a single block.

3.2.1 Question 1: How do the execution times of matrix multiplication compare when employing the three different memory types?

First off in figure 4 we can see that the constant memory implementation is the slowest, this is a bit odd as it is optimised for read-only access so one would expect it to be faster than global memory. This can possibly be explained because of uncoalesced memory access. It is however logical to be slower than shared memory since shared memory is per block and on chip, whereas constant memory is off chip.

3.2.2 Question 2: How do the times scale with input and/or grid size?

As size increases, global memory time scales up dramatically due to bandwidth limits, while constant memory spikes much earlier due to its tiny cache. The shared memory graph has the fastest operation speed, but also has the smallest buffer size, meaning that we should only copy the row and column used for the multiplication of one cell in shared memory. This is one way to continue using the faster shared memory and avoid clashing with the small buffer size. We did not implement this in our code, we decided to store all three matrices ($r = a * b$) in shared memory, resulting in the earlier stop in measurement data.

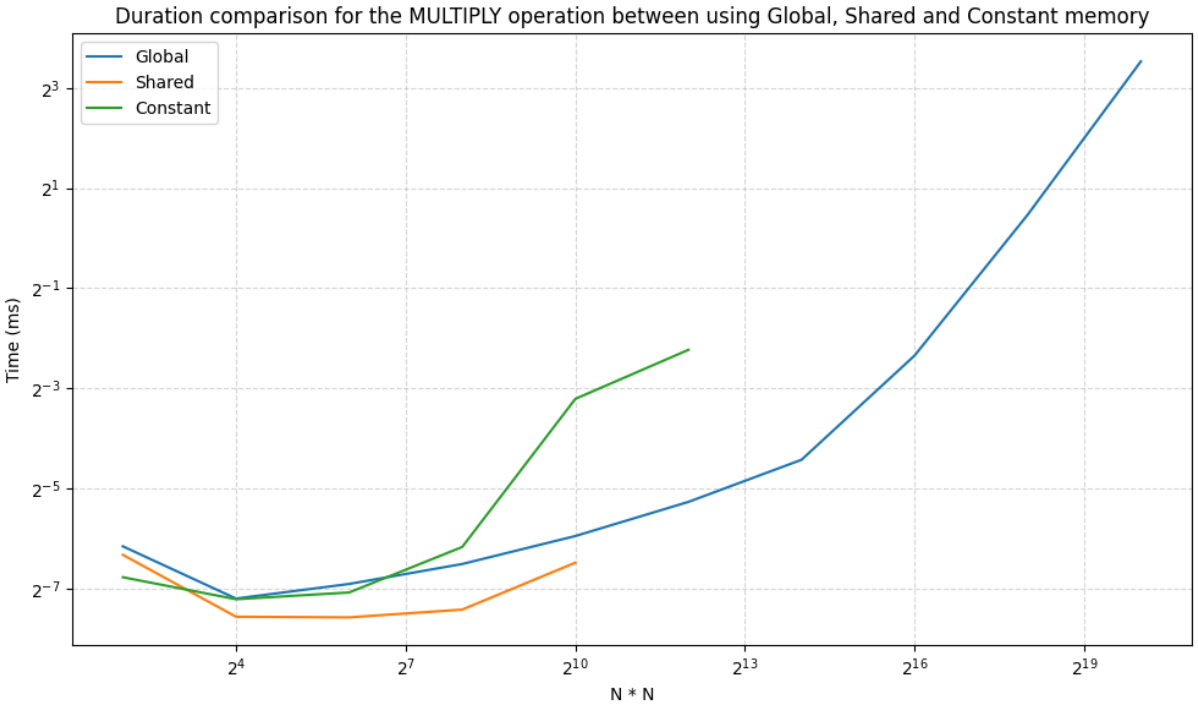


Figure 4: Comparison between different memory types used

4 Conclusion

In this lab we have seen the importance of coalesced memory access, as well as the differences between global, shared and constant memory. We have seen that for larger matrix sizes the GPU is much faster than the CPU, this is mostly due to the fact that the GPU can perform many operations in parallel. We also saw that the copy operation is faster than the transpose operation, this is mostly due to coalesced memory access but also because the copy operation is an easier operation than the transpose operation. We also created an implementation of matrix multiplication using global, shared and constant memory, we saw that the shared memory implementation was the fastest, this is mostly due to the fact that shared memory is on chip and per block, whereas global memory is off chip and constant memory is off chip and optimised for read-only access.

5 Appendix

In listing 6 we show the code we used for part 1 of this lab exercise, for part 2 of this lab exercise we used similar code with small adjustments in order to adjust for the change in memory type.

Listing 6: CUDA code for constant memory multiplication

```
int main (void) {
    float *d_a, *d_r;
    int N = 2;

    std::ofstream csv_file("....csv");
    csv_file << "bytes,ms\n";


    int MAX_BLOCKS = 5;
    int BLOCKS = 10;

    int p = 1;
    int p_max = 9;

    int THREADS = 1024;
    int COUNT = 20;

    for ( ; p <= p_max; p++) {
        N = pow(2, p);
        float total_ms = 0;

        // Create matrix
        std::vector<float> matrix;
        srand((unsigned)time(NULL));

        for (int i = 0; i < N * N; i++){
            float randomValue = static_cast<float>(std::rand()) /
                RAND_MAX * 100; // Generates between 0 and 100
            randomValue = std::round(randomValue * 100.0f) /
               100.0f;
            matrix.push_back(randomValue);
        }

        // Allocate output
        std::vector<float> res(N * N);

```

```
35          // Allocate Device memory
36          cudaMalloc((void**)&d_a, N * N * sizeof(float));
37          cudaMalloc((void**)&d_r, N * N * sizeof(float));
38
39          cudaMemcpy(d_a, matrix.data(), N * N * sizeof(float),
                cudaMemcpyHostToDevice);
40
41          // take an average
42          for (int i = 0; i < COUNT; i++) {
43              // Start Timing
44              cudaEvent_t start_cuda, stop_cuda;
45              cudaEventCreate(&start_cuda);
46              cudaEventCreate(&stop_cuda);
47              cudaEventRecord(start_cuda);
48
49              // Run the Device code
50              cuda_method<<<BLOCKS, THREADS>>>(d_a, d_r, N * N, N);
51
52              // Stop timing
53              cudaEventRecord(stop_cuda);
54              cudaEventSynchronize(stop_cuda);
55              float ms;
56              cudaEventElapsedTime(&ms, start_cuda, stop_cuda);
57
58              total_ms += ms;
59          }
60
61          float avg_ms = total_ms / COUNT;
62          int strides = (N*N) / (BLOCKS * THREADS);
63
64          float bytes_transferred = N * N * 4;
65          double bw = (bytes_transferred / (avg_ms * pow(10,6)));
66
67          write_to_csv(csv_file, bytes_transferred, avg_ms);
68          std::cout << "Avg␣duration␣for␣" << BLOCKS << "␣blocks,␣"
                << THREADS << "␣size:␣" << N << "␣and␣equals␣a␣
               bandwidth␣of␣" << bw << "GB/s␣and␣took␣" << avg_ms << "
               ␣ms" << std::endl;
69      }
70
71
72      cudaFree(d_a);
73      cudaFree(d_r);
```

```
74
75      return 0;
76  }
```